

1. Принятые решения

Хеширование (фильтр Блюма + CMS)

- Используется 2 основные хеш-функции: `xxhash` и `murmur3`
- Применяется схема double-hashing:
 1. на каждом уровне для ключа 2 хеша (h_1 и h_2) - с помощью первой и второй хеш-функций соответственно
 2. хеш ключа на i -ом уровне вычисляется как

$$h = h_1 + (1 + i) \cdot h_2$$

3. таким образом, на каждом уровне имеем по сути отдельную хеш-функцию

Фильтр Блюма

Бинарные значения ячеек фильтра хранятся в виде отдельных битов:

1. "под капотом" Фильтр хранит $\text{size}/8 + 1$
2. при изменении/получении ячейки с индексом i вычисляется байт, в котором находится соответствующий бит, а далее с помощью битовых сдвигов изменяется/достаётся значение ячейки
3. это позволяет занимать в 8 раз меньше памяти, чем при хранении "в лоб" (1 ячейка - 1 байт)

2. Подбор параметров

Фильтр Блюма

$$c_{\text{best}} = \left\lceil \frac{\log_{10} \frac{1}{P(FP)}}{(\ln 2)^2} \right\rceil \approx \left\lceil 2 \log_{10} \frac{1}{P(FP)} \right\rceil$$
$$k_{\text{best}} = \lceil c \ln 2 \rceil \approx \lceil 0.7c \rceil$$

где

- $P(FP)$ - вероятность ложно-положительного срабатывания
- c - `size`
- k - `hashes`

Count-Min-Sketch

$$w_{\text{best}} = \frac{2}{\epsilon}$$

$$d_{\text{best}} = \log_2 \frac{1}{p}$$

где

- ε - предел превышения реальной частоты (если $\varepsilon = 0.5$, то допускаем превышение вычисляемой частоты на 50% относительно реальной)
- p - вероятность, с которой в среднем будет превышение частоты на ε и больше
- w - width
- d - depth

3. Контроль (как проверялось)

Фильтр Блюма

Были добавлены тесты на:

- корректность работы пустого Фильтра Блюма (в котором нет ключей)
- корректность работы при вставке одного ключа
- корректность работы при вставке многих ключей

Также были добавлены 3 бенчмарка:

- DDOS - отфильтровываем 10K "мошеннических" ключей после вставки 25M "обычных" ключей и проверяем, сколько мошенников обнаружит Фильтр
- WIDTH - при `hashes=1` измеряем, как меняется производительность при вставке 10M ключей в зависимости от значений `size`
- DEPTH - для нескольких значениях `size` измеряем, как меняется производительность при вставке 10M ключей в зависимости от значений `hashes`

Count-Min-Sketch

Были добавлены тесты на:

- корректность работы пустой структуры (в которой нет ключей)
- корректность работы при вставке одного ключа
- корректность работы при вставке многих ключей (последовательность + рандомно)

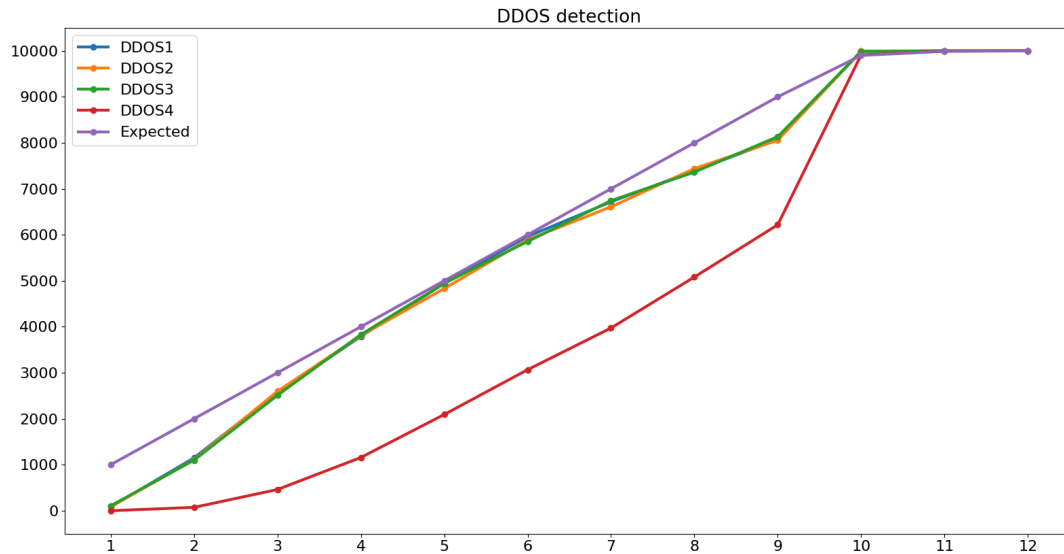
Также был добавлен бенчмарк, в котором для различных распределений 10K ключей по частотам при различных параметрах оцениваются частоты ключей.

4. Результаты бенчмарок

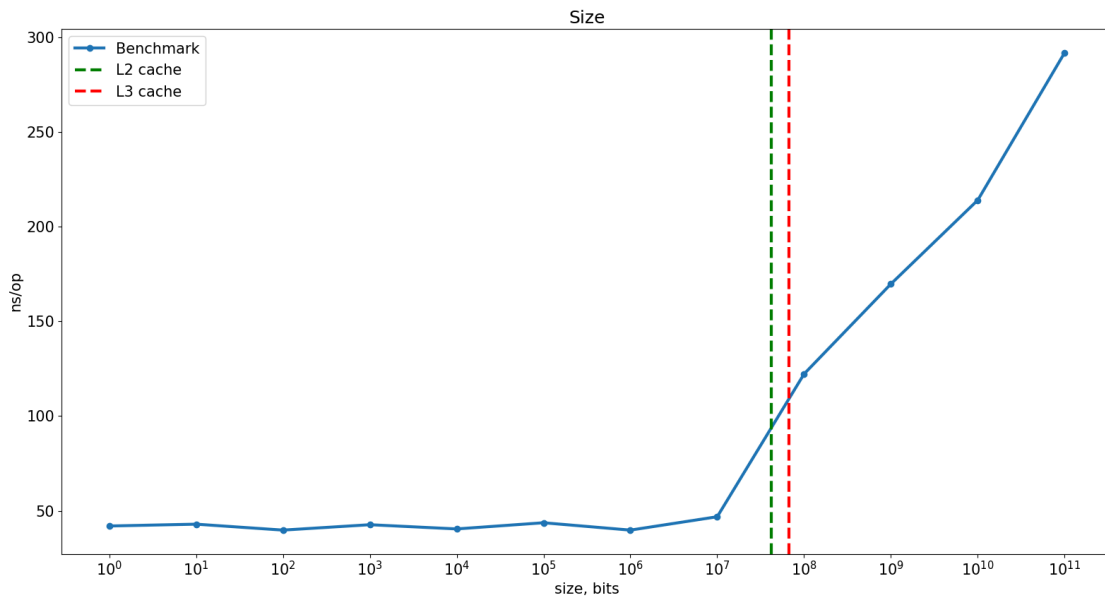
Bloom_DDOS

Бенчмарк запускался при различных сочетаниях "обычных" и "мошеннических" ключей:

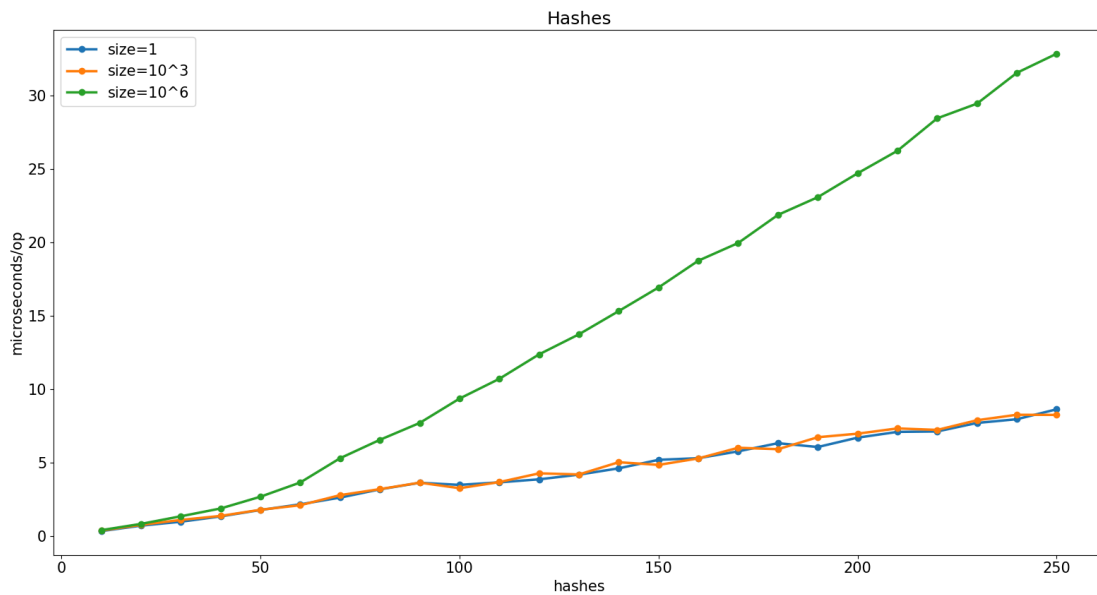
1. Обычные ключи - `user_%d_not_fraud` , мошеннические - `user_%d` , при этом в первом случае `%d` - от 0 до 24999999, а во втором - от 0 до 9999
2. Обычные ключи - `user_%d_not_fraud` , мошеннические - `user_%d` , при этом в первом случае `%d` - от 0 до 24999999, а во втором - от 25000000 до 25009999
3. Обычные ключи - `user_%d` , мошеннические - `user_%d` , при этом в первом случае `%d` - от 0 до 24999999, а во втором - от 25000000 до 25009999
4. Все ключи - UUID4



Bloom_SIZES



Bloom_HASHES



Count-Min-Sketch

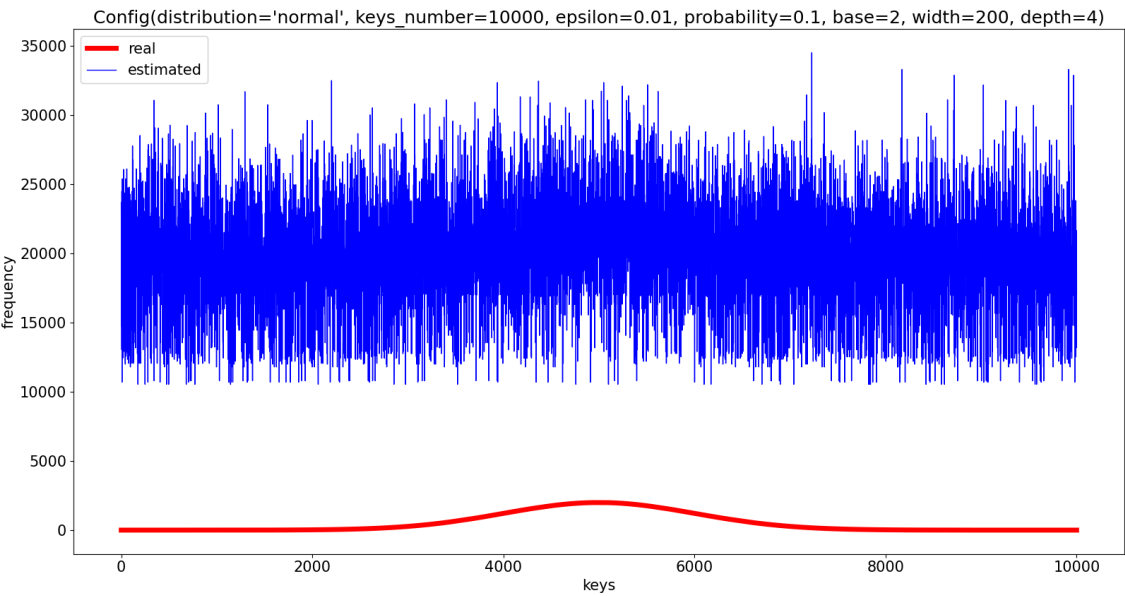
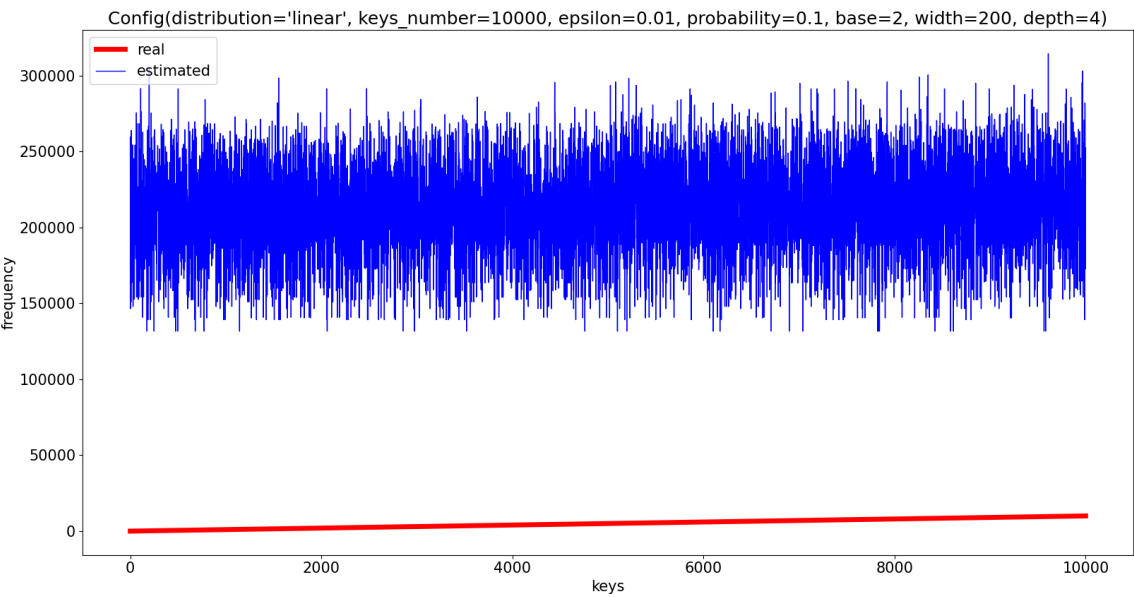
Конфигурация

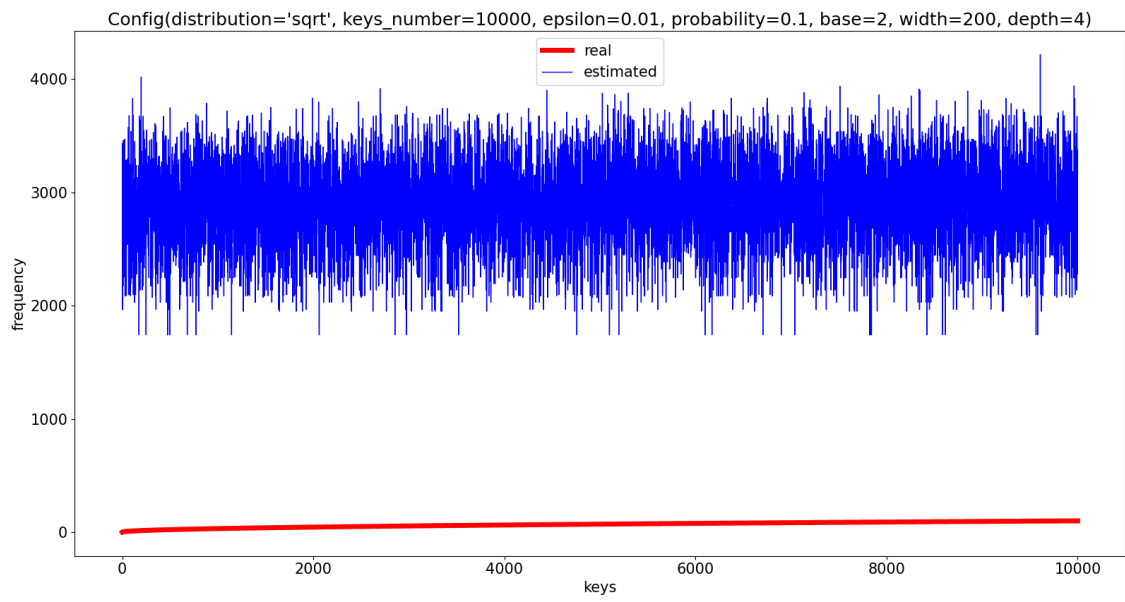
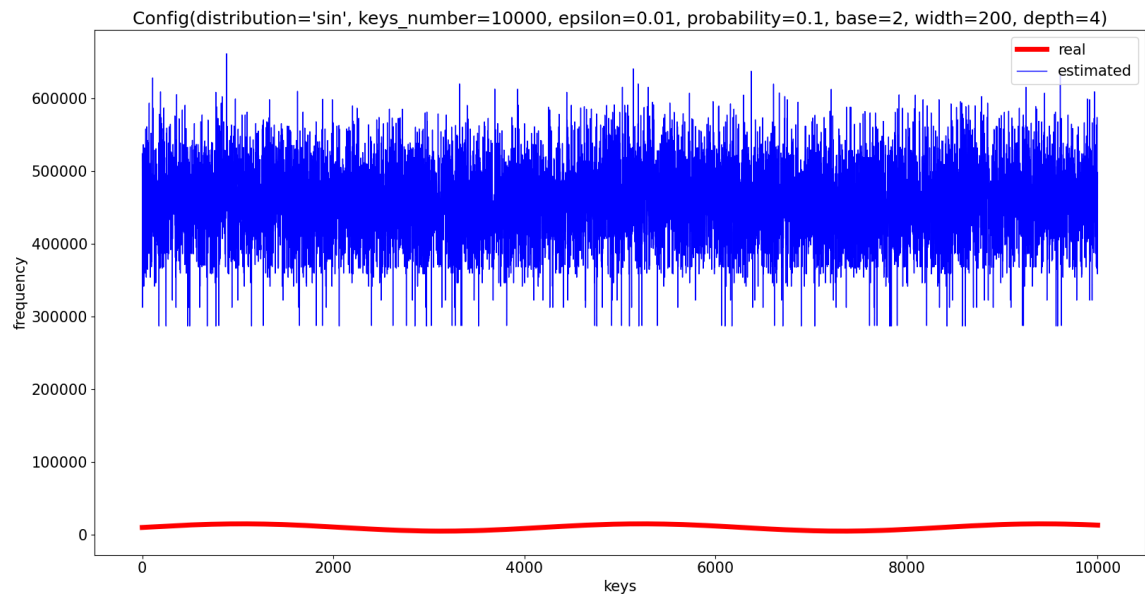
В бенчмарке использовалось 10K ключей типа UUID4, поэтому для полного хранения всего потока в худшем случае требовалось бы $10 \cdot 1024 \cdot 16 \cdot 8 = 1.25$ Мбайт

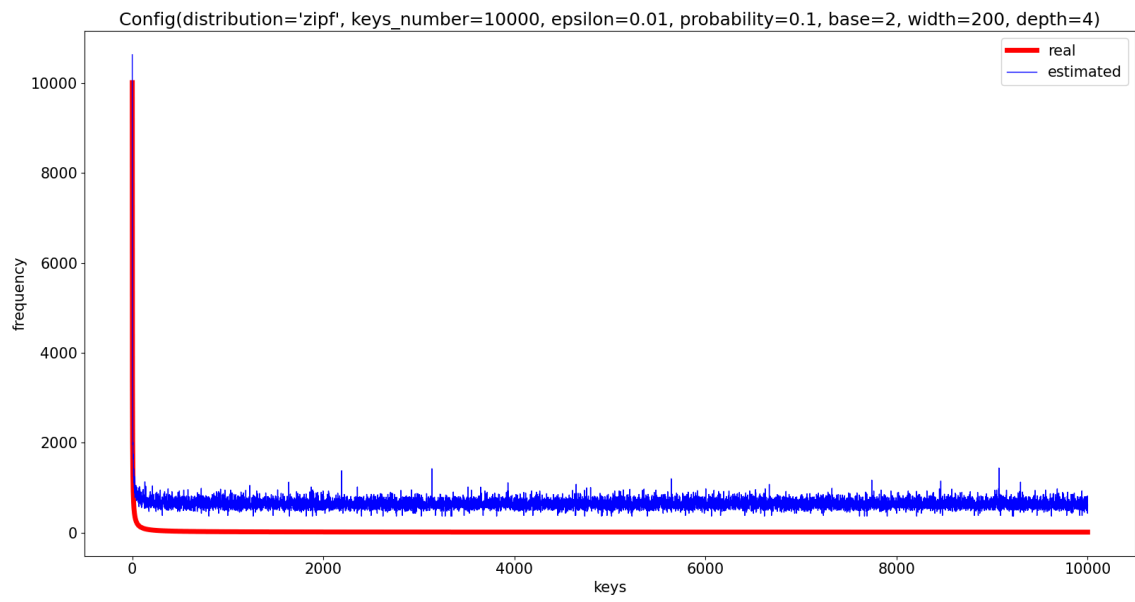
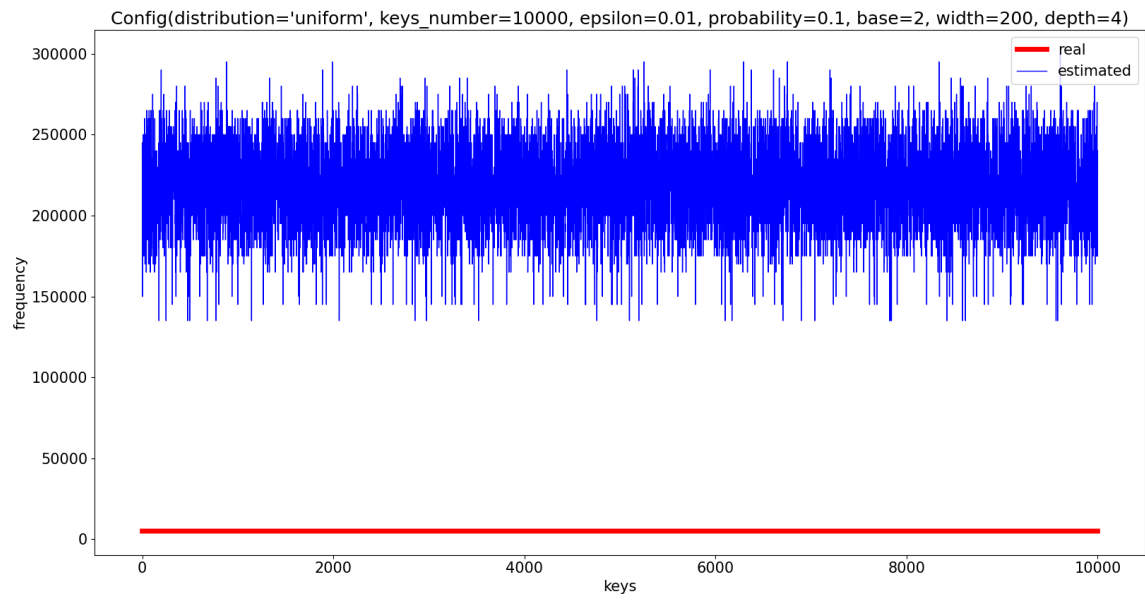
Бенчмарк запускался при 4 разных конфигурациях (в зависимости от параметров конфигурации вычислялись значения `width` и `depth`):

1. $\varepsilon = 0.01$, $p = 0.1$ (6.25 Кбайт)
2. $\varepsilon = 0.001$, $p = 0.01$ (110 Кбайт)
3. $\varepsilon = 0.0001$, $p = 0.01$ (1.1 Мбайт)
4. $\varepsilon = 0.0001$, $p = 0.001$ (1.5 Мбайт)

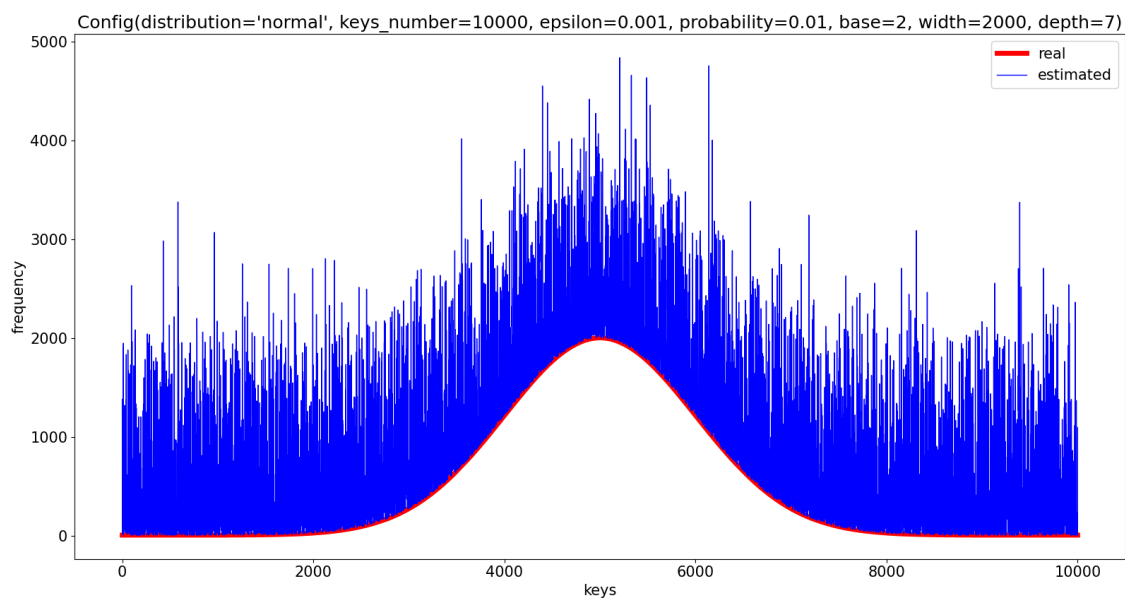
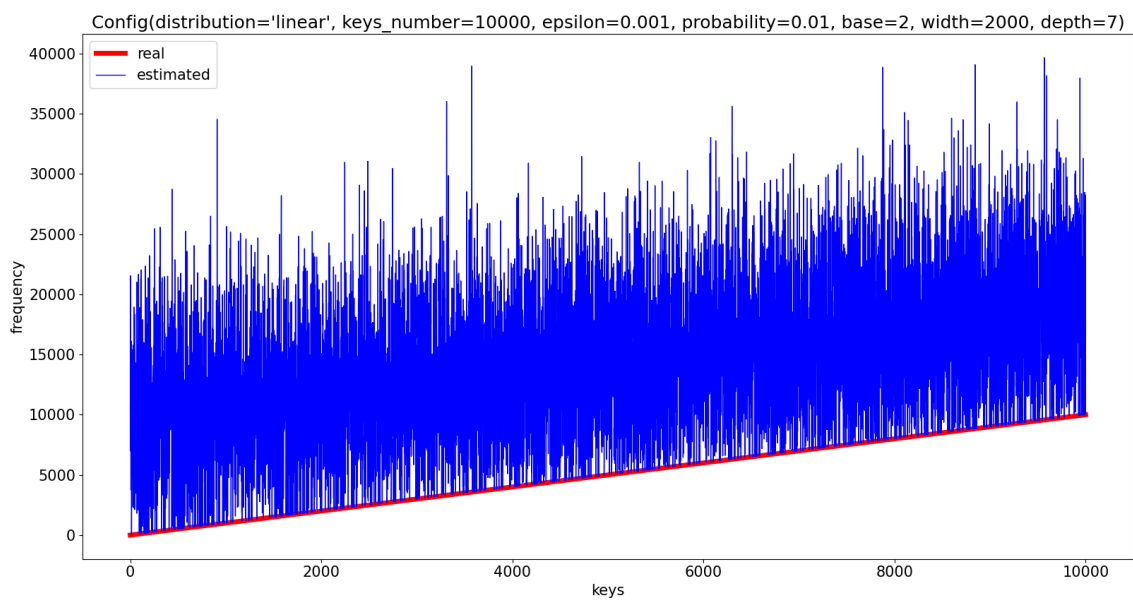
Конфигурация 1

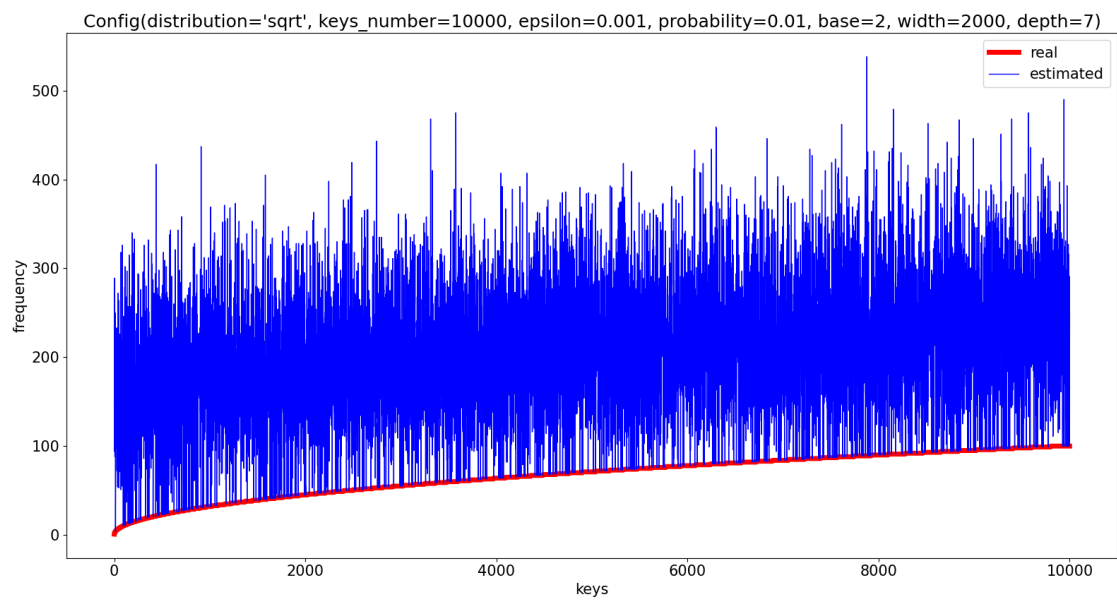
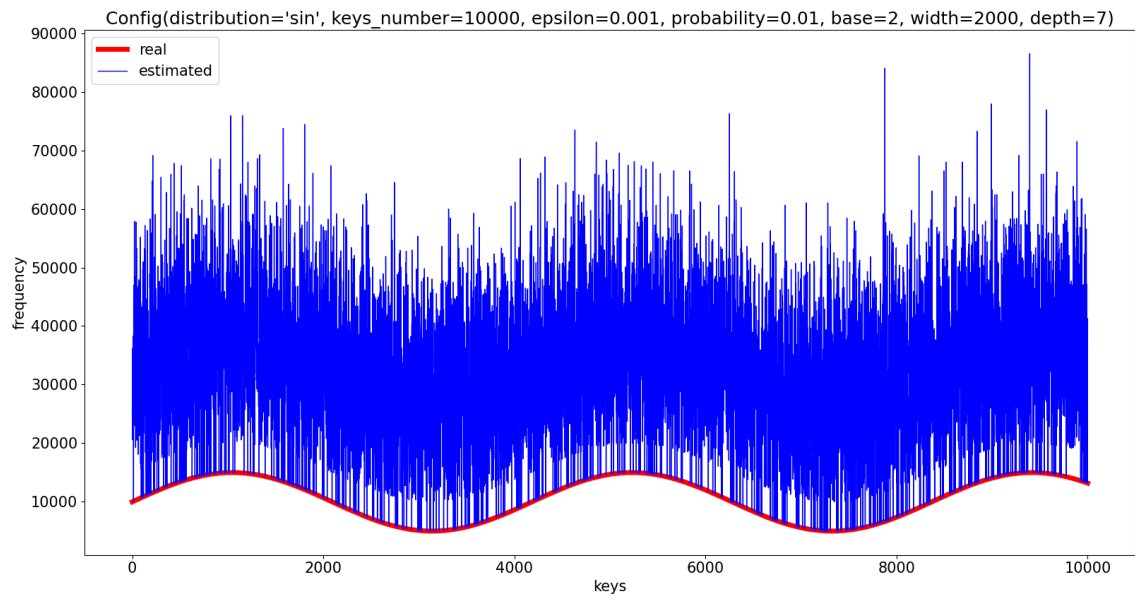


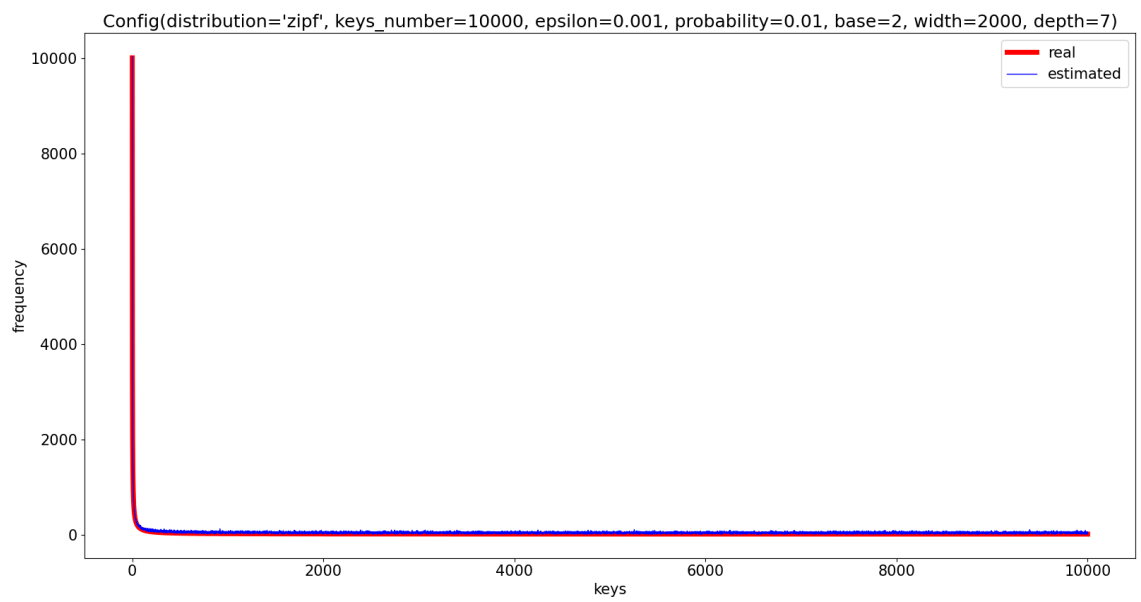
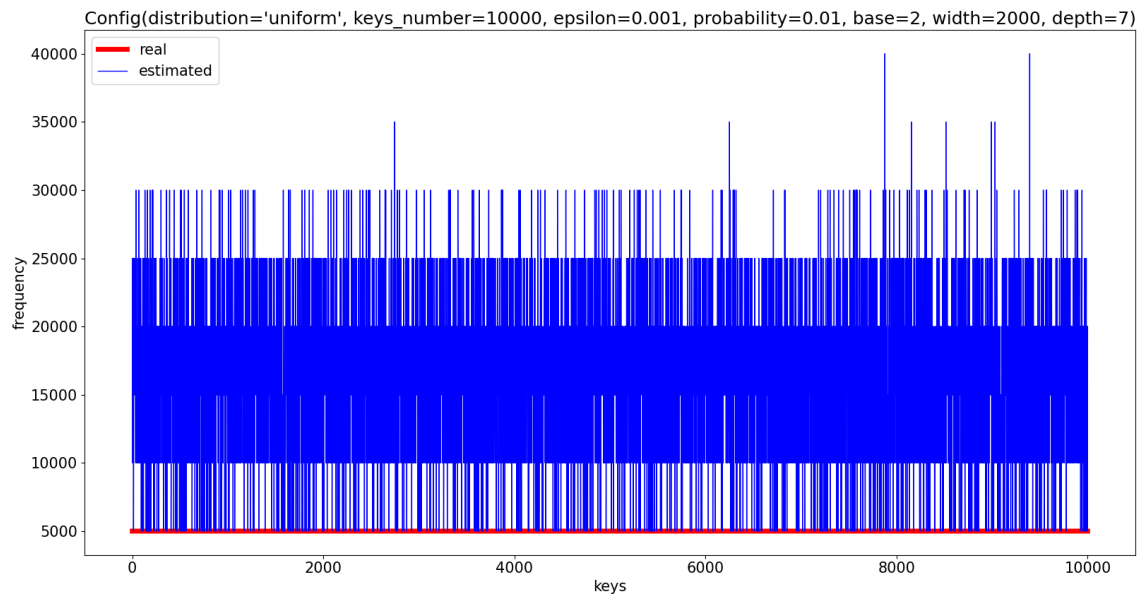


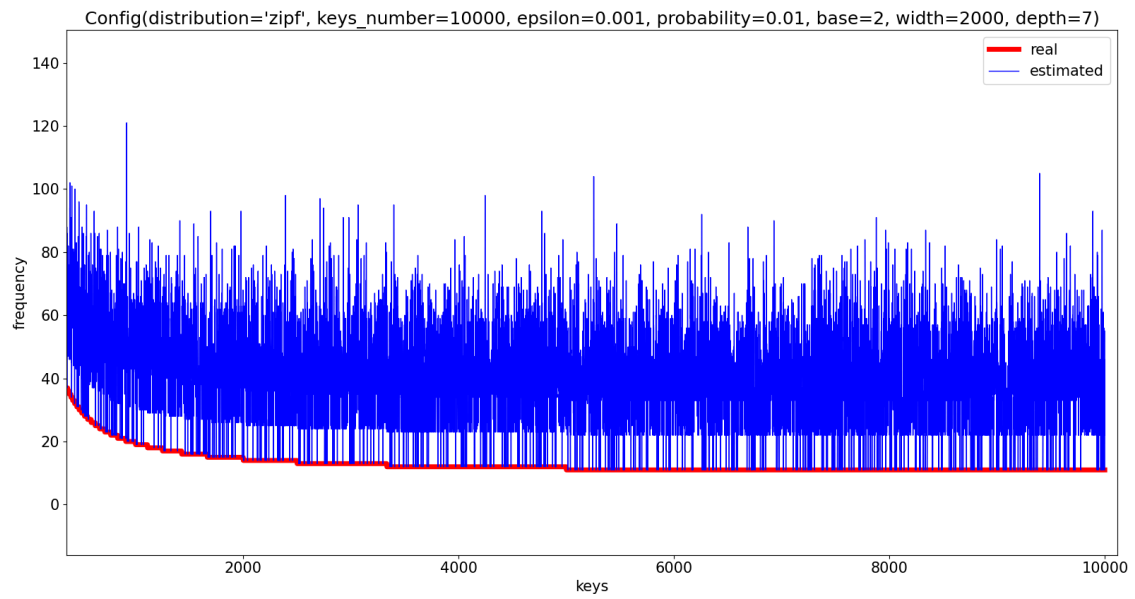


Конфигурация 2

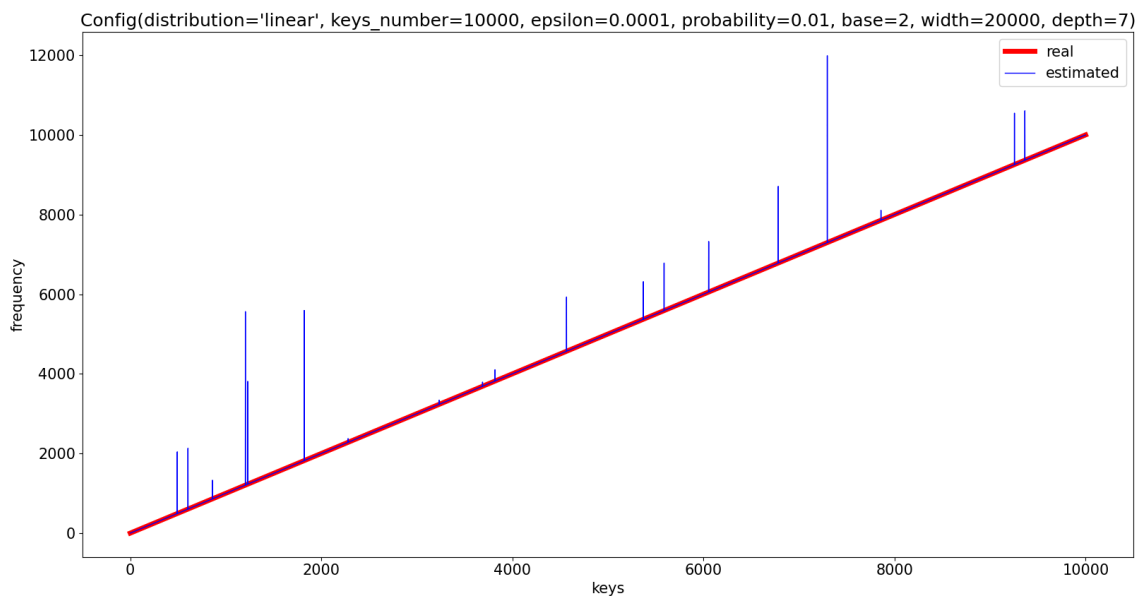


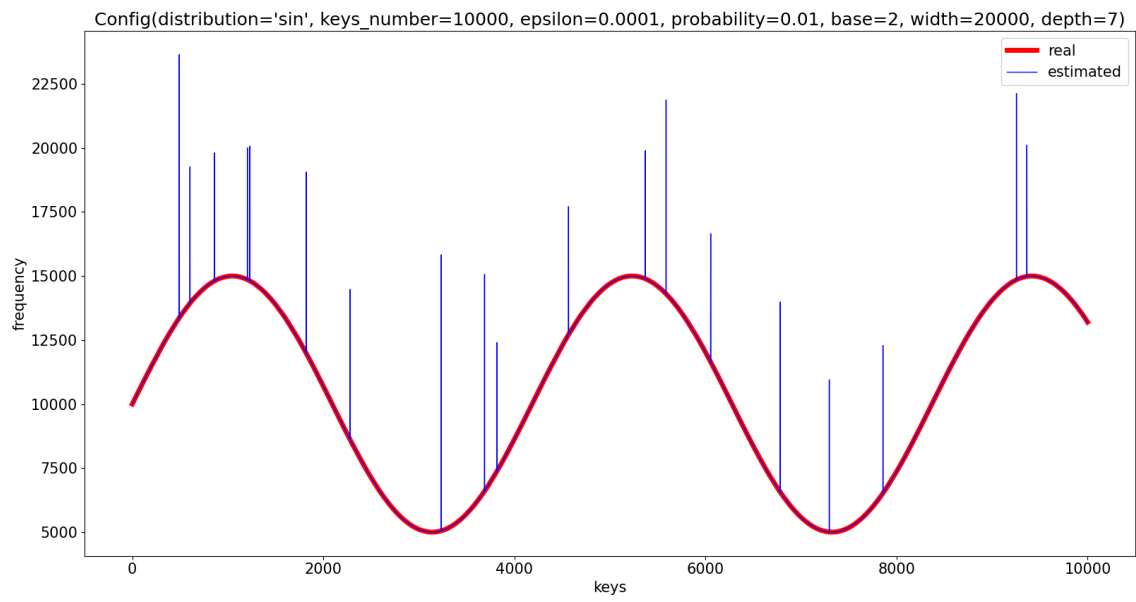
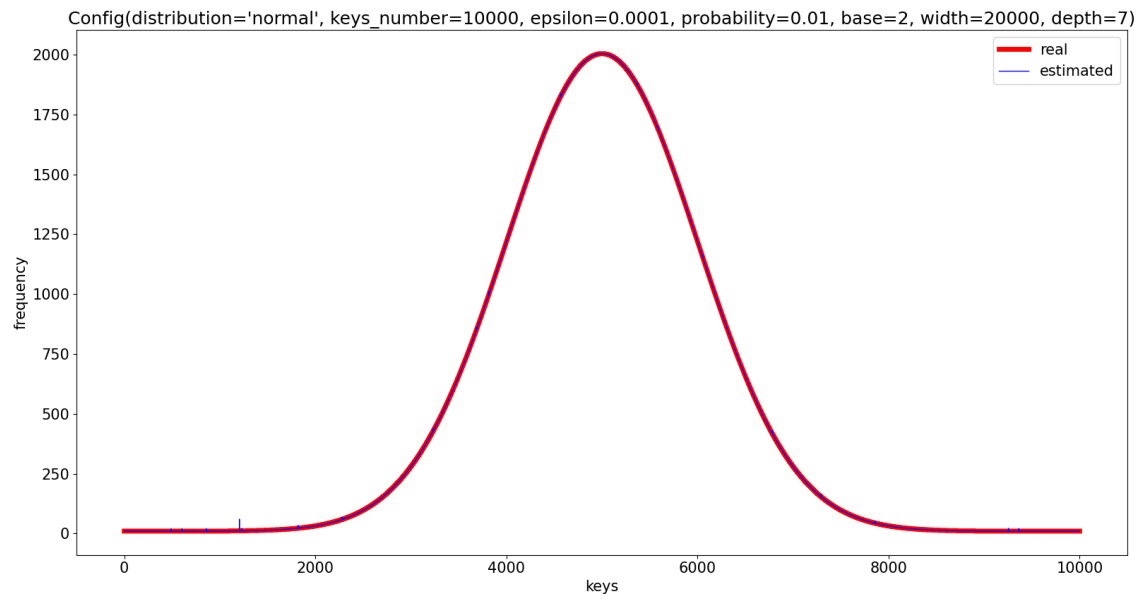


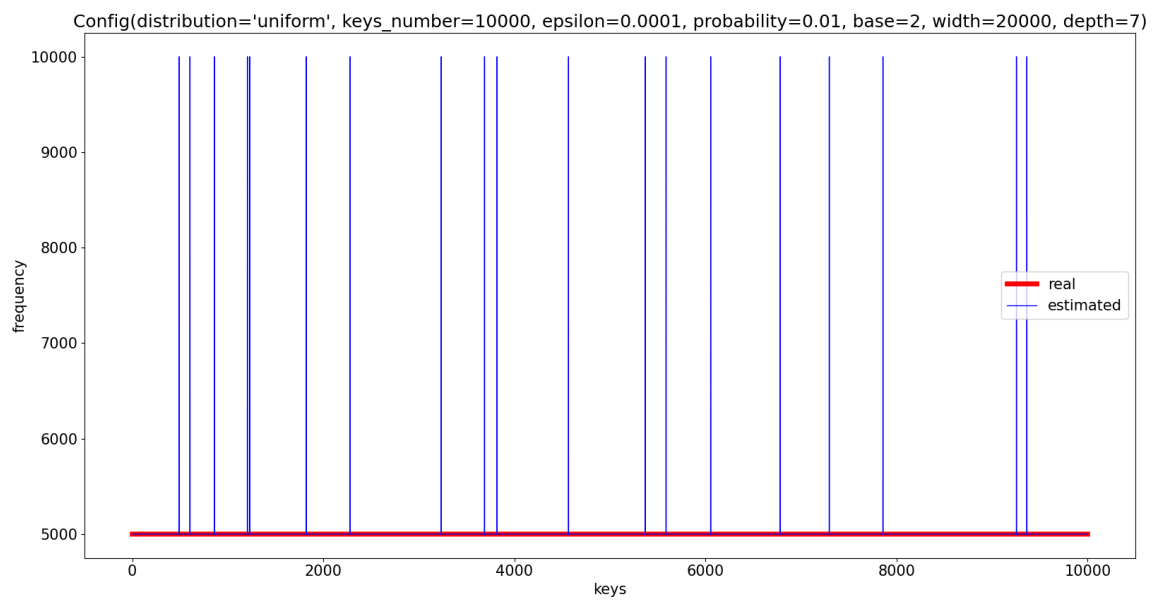
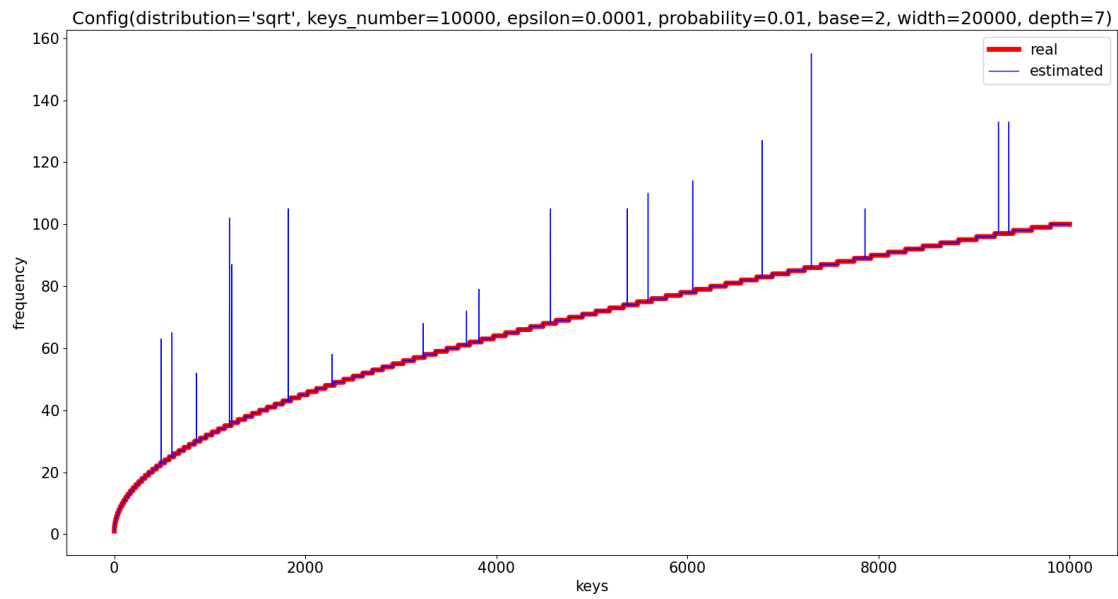


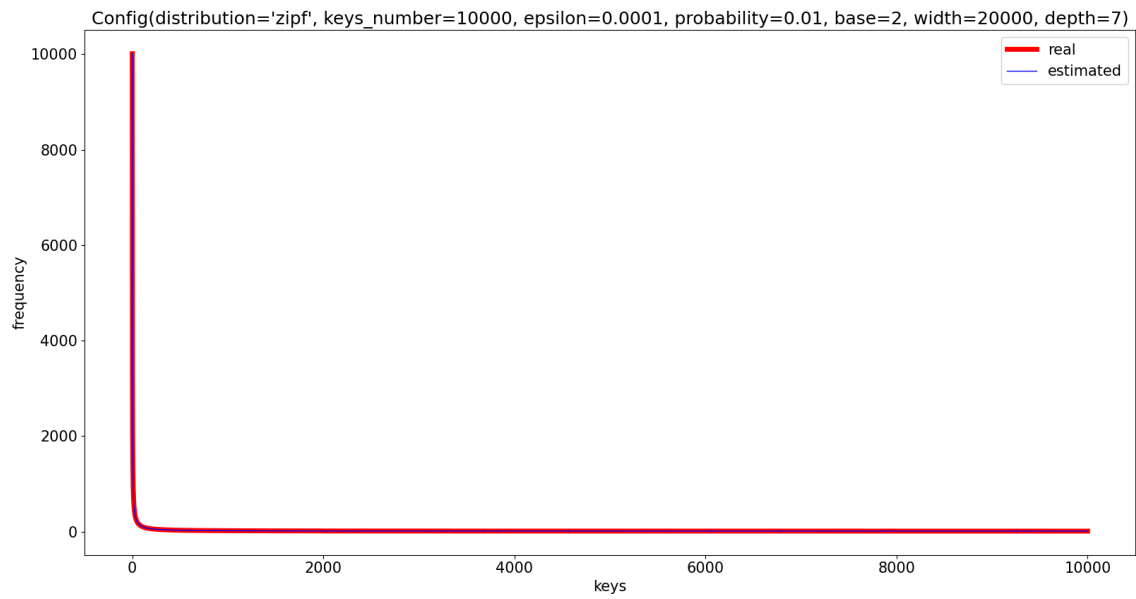


Конфигурация 3

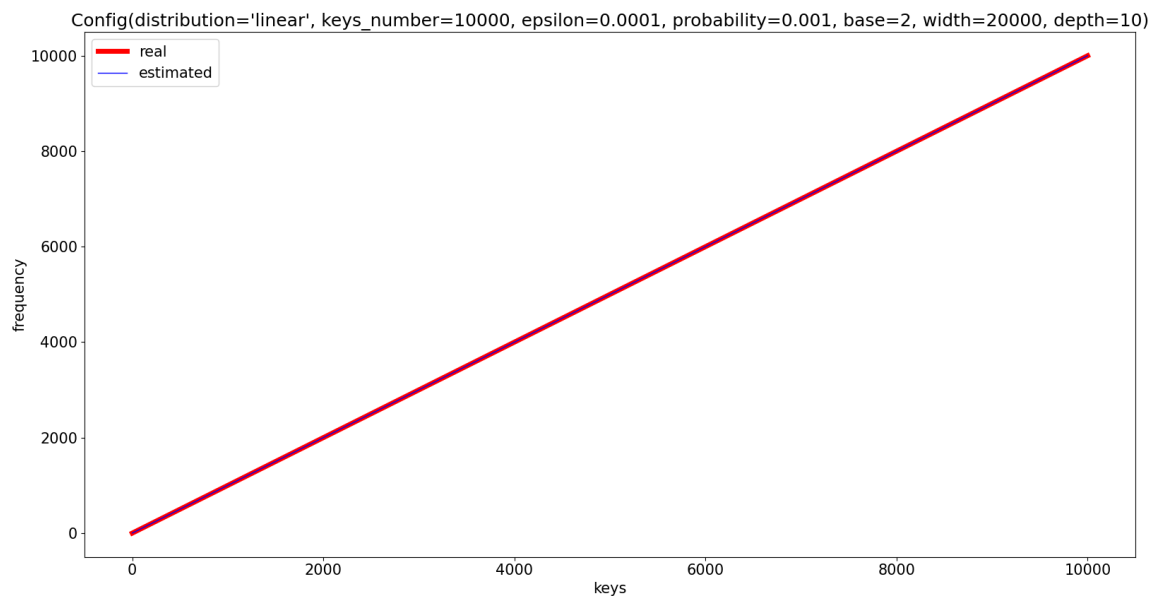


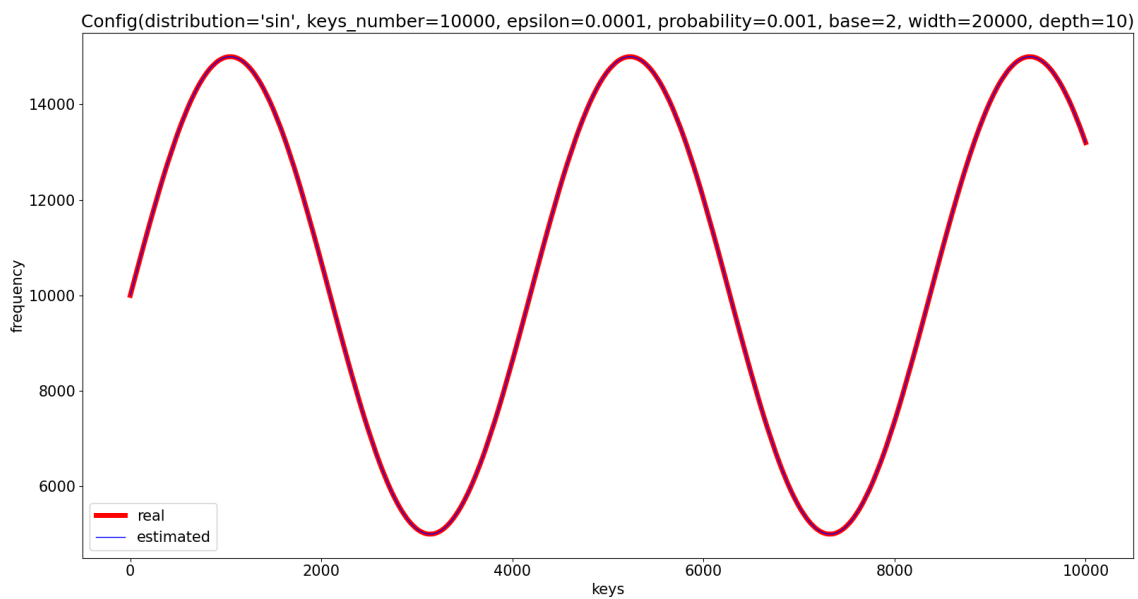
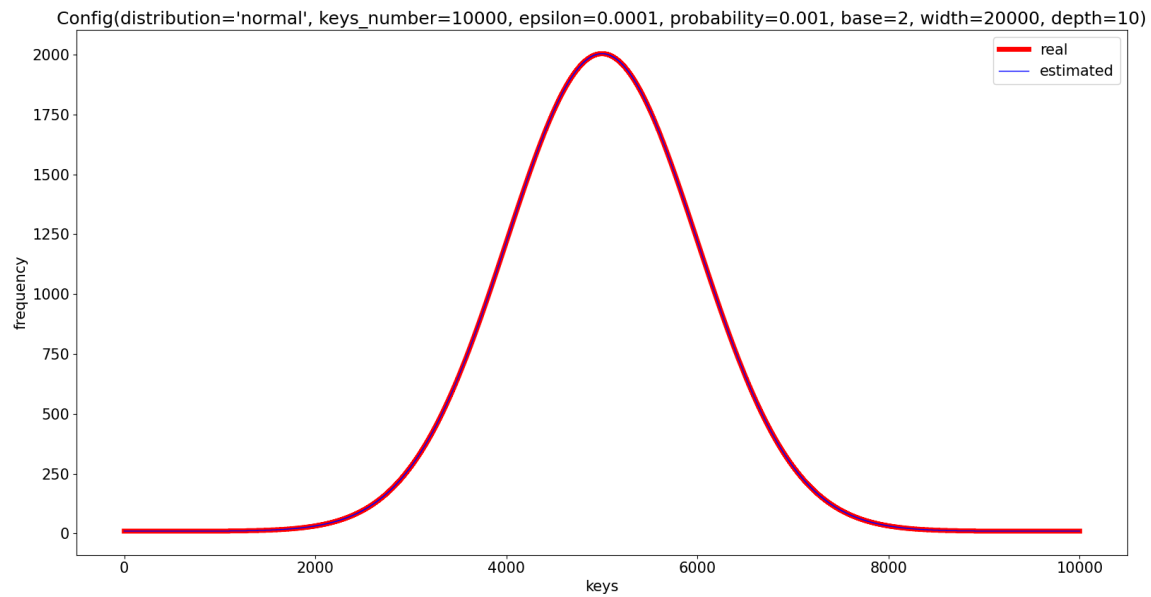


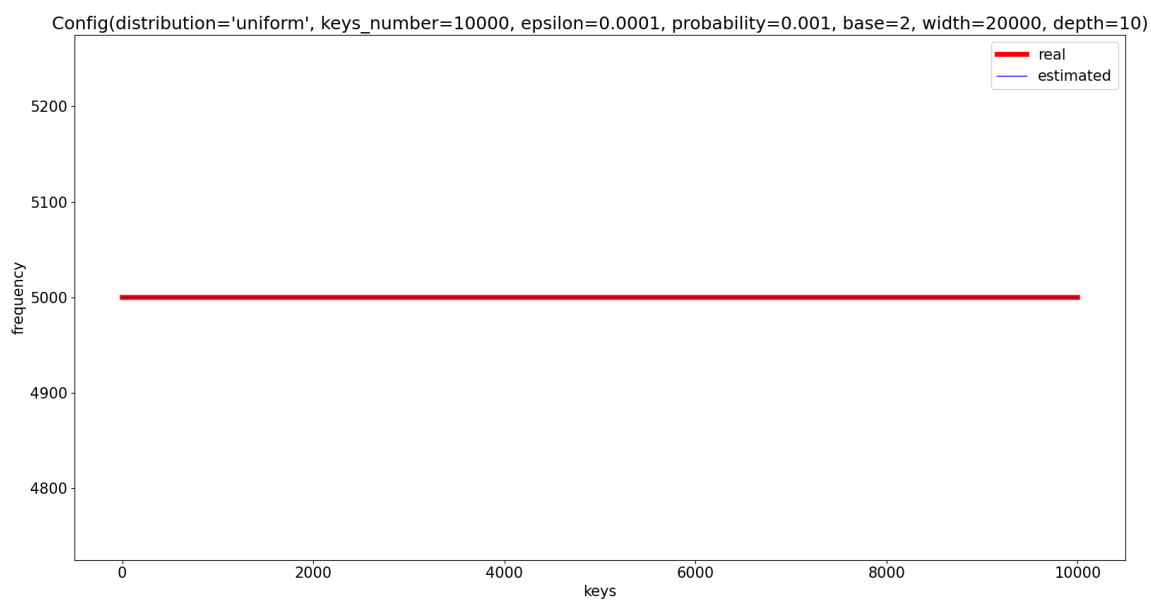
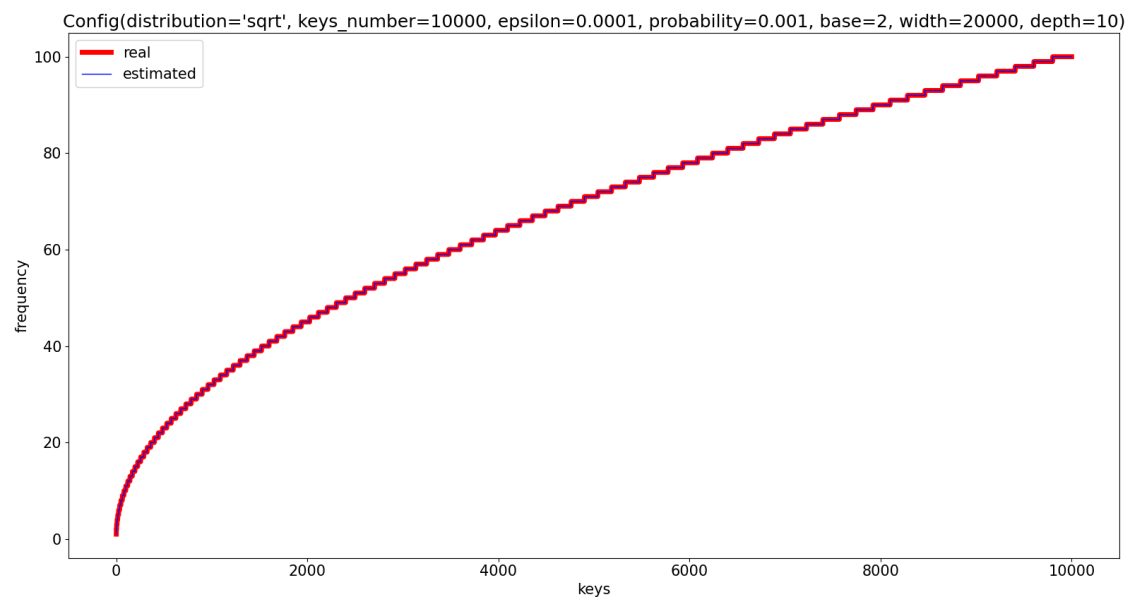


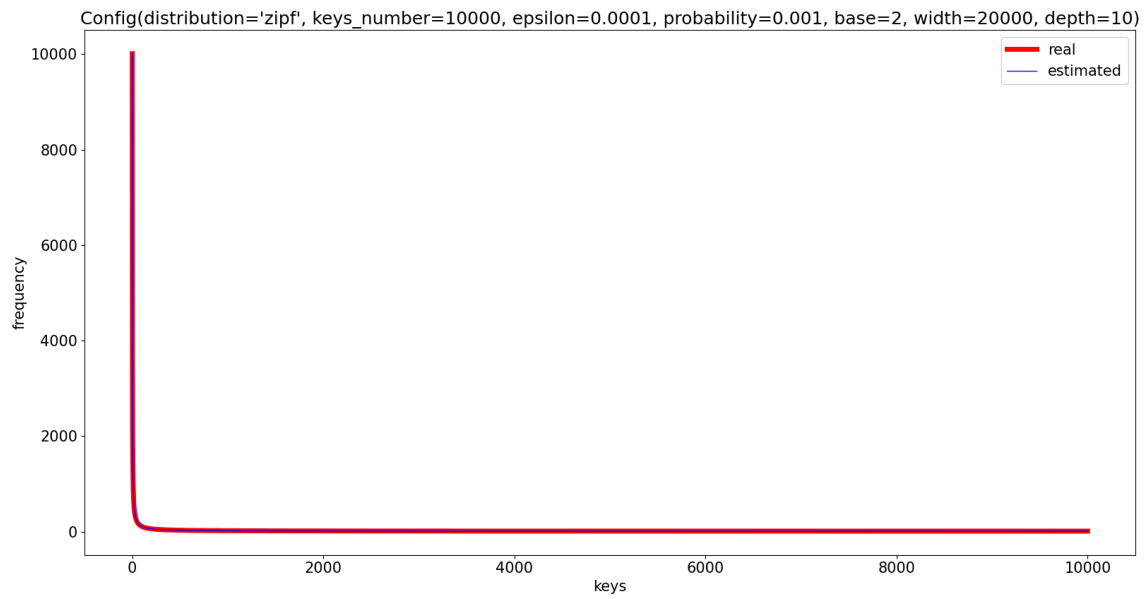


Конфигурация 4

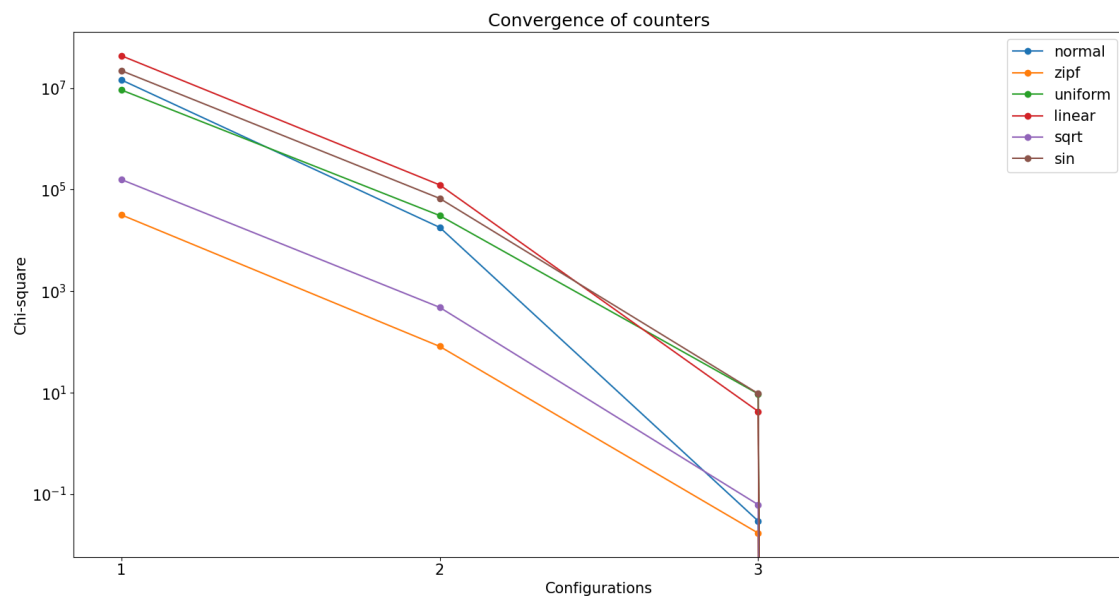








Сходимость по всем конфигурациям



На четвёртой конфигурации ошибка χ^2 равняется нулю для всех распределений.